

RESEARCH PAPER

Optimizing Serverless Function Placement in Edge-Cloud Continuum Architectures

Authors: Dr. Vint Cerf, Prof. Leslie Lamport, Dr. Shafi Goldwasser

ABSTRACT

Serverless computing paradigms are increasingly being extended from centralized cloud platforms to geographically distributed edge infrastructure. This paper addresses the function placement optimization problem in edge-cloud continuum environments, where latency-sensitive workloads must be dynamically allocated across heterogeneous computing nodes. We formulate the placement decision as a constrained optimization problem and propose a reinforcement learning-based scheduler that reduces average end-to-end latency by 41 percent compared to round-robin baselines while maintaining resource utilization above 82 percent. Evaluation on a 12-node testbed with realistic microservice workloads validates the practical applicability of our approach.

Keywords: Serverless Computing, Edge Computing, Function Placement, Reinforcement Learning

1. INTRODUCTION

The evolution of cloud computing toward edge-cloud continuum architectures demands new approaches to workload orchestration. Traditional serverless platforms such as AWS Lambda and Azure Functions operate within centralized data centers, introducing latency penalties for geographically distributed applications. Edge computing promises sub-10ms response times by processing data closer to its source, but heterogeneous resource availability and dynamic demand patterns complicate scheduling decisions.

2. PROBLEM FORMULATION

We model the edge-cloud continuum as a weighted graph $G = (V, E)$ where vertices represent computing nodes with heterogeneous CPU, memory, and GPU capacities, and edges represent network links with measured latency and bandwidth characteristics. The function placement problem seeks to assign incoming serverless invocations to computing nodes that minimize aggregate response latency subject to resource capacity constraints and data locality preferences.

3. PROPOSED APPROACH

Our reinforcement learning scheduler employs a Proximal Policy Optimization agent that observes system state vectors comprising current node utilizations, pending request queues, and historical latency measurements. The action space corresponds to node selection decisions, and the reward signal combines negative latency with resource balance penalties. We train the agent using simulated workload traces derived from Azure Function execution logs.